

EXPERIMENT 3

DEBUGGING TOOLS IN C

PROGRAMMING

Name: Shashwat Raj Singh

SAP ID: 590033211

Batch: B-16

Program: B.Tech Computer Science and Engineering

Aim

To understand different types of errors in C and learn systematic debugging using compiler diagnostics, warning flags, printf-based tracing, and GDB.

Basic Concepts

Error: A mistake made by the programmer in the source code.

Bug: A flaw causing incorrect or unexpected behavior.

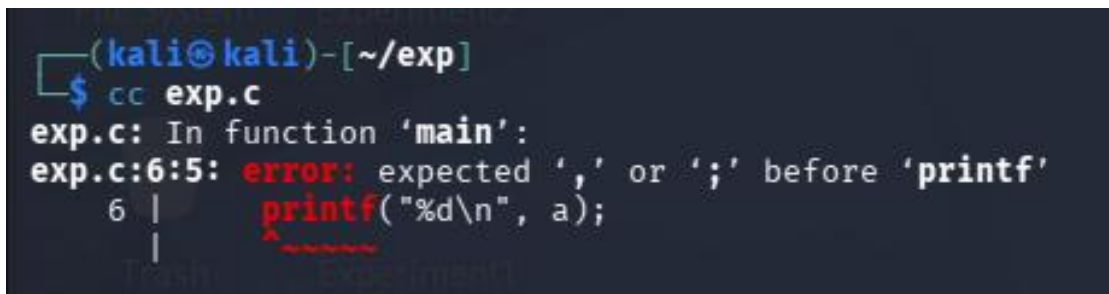
Debugging: The systematic process of finding and removing bugs.

Case 1

Code:

```
#include <stdio.h>
int main(){
    int a=10
    printf("%d\n",a);
    return 0;
}
```

Screenshots:



```
(kali@kali)-[~/exp]
$ cc exp.c
exp.c: In function 'main':
exp.c:6:5: error: expected ',' or ';' before 'printf'
  6 |     printf("%d\n", a);
    |     ^~~~~~
```

The screenshot shows a terminal window with a dark background. The prompt is `(kali@kali)-[~/exp]`. The user has entered `$ cc exp.c`. The compiler output shows an error in `exp.c` at line 6, column 5: `error: expected ',' or ';' before 'printf'`. The corresponding line of code is `printf("%d\n", a);`, with a red arrow pointing to the opening curly brace of the `printf` function call.

```
(kali@kali)-[~/exp]
$ cc exp.c

(kali@kali)-[~/exp]
$ ./a.out
10
```

Explanation:

Missing semicolon before printf caused a syntax error. After adding the semicolon, the program compiled and printed 10.

Case 2

Code:

```
#include <stdio.h>
int main(){
    int a;
    printf("Hello\n");
    return 0;
}
```

Screenshots:

```
(kali@kali)-[~/exp]
$ cc exp1.c

(kali@kali)-[~/exp]
$ ./a.out
Hello
```

```
(kali@kali)-[~/exp]
$ cc -Wall -Wextra exp1.c
exp1.c: In function 'main':
exp1.c:5:9: warning: unused variable 'a' [-Wunused-variable]
     5 |     int a;
       |         ^
```

Explanation:

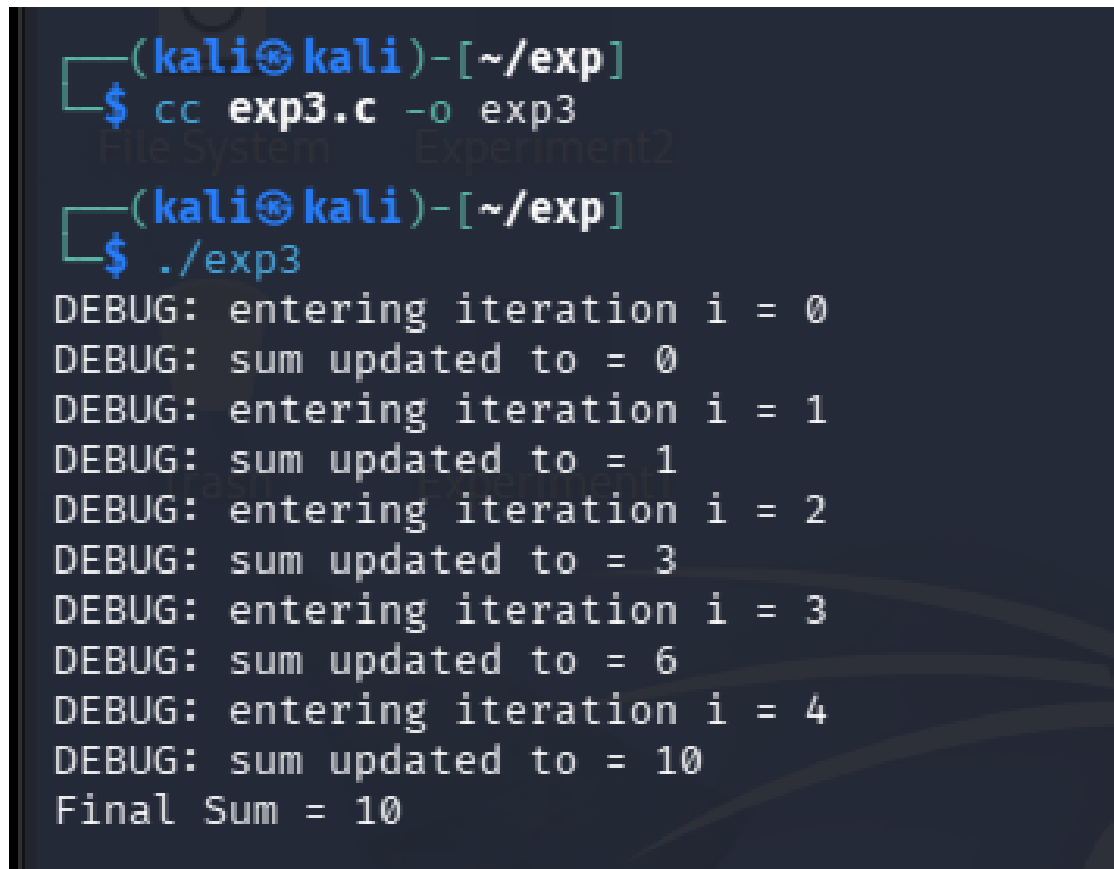
The compiler warned that variable 'a' was declared but never used. Removing the unused variable resolves the warning.

Case 3

Code:

```
#include <stdio.h>
int main(){
    int sum=0;
    for(int i=0;i<5;i++){
        printf("DEBUG: entering iteration i = %d\n",i);
        sum+=i;
        printf("DEBUG: sum updated to %d\n",sum);
    }
    printf("Final Sum = %d\n",sum);
}
```

Screenshots:



```
(kali㉿kali)-[~/exp]
$ cc exp3.c -o exp3
(kali㉿kali)-[~/exp]
$ ./exp3
DEBUG: entering iteration i = 0
DEBUG: sum updated to = 0
DEBUG: entering iteration i = 1
DEBUG: sum updated to = 1
DEBUG: entering iteration i = 2
DEBUG: sum updated to = 3
DEBUG: entering iteration i = 3
DEBUG: sum updated to = 6
DEBUG: entering iteration i = 4
DEBUG: sum updated to = 10
Final Sum = 10
```

```
(kali@kali)-[~/exp]
$ cc exp3.c -o exp3

(kali@kali)-[~/exp]
$ ./exp3
Final Sum = 15
```

Explanation:

printf debugging showed loop iterations and sum updates. Correcting the loop to include 5 produced Final Sum = 15.

Case 4

Code:

```
#include <stdio.h>
int main() {
    int i, sum=0;
    for(i=1; i<5; i++)
        sum=sum+i;
    printf("Sum = %d\n", sum);
}
```

Screenshots:

```

(gdb) list
4      {
5          int i, sum = 0;
6
7      for(i = 1; i < 5; i++)
8      {
9          sum = sum + i;
10     }
11
12     printf("Sum = %d\n", sum);
13
(gdb) break 8
Note: breakpoint 1 also set at pc 0x55555555151.
Breakpoint 2 at 0x55555555151: file exp4.c, line 9.
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/exp/exp4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at exp4.c:9
9          sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at exp4.c:9
9          sum = sum + i;
1: i = 2
2: sum = 1

```

```

(gdb) continue
Continuing.

Breakpoint 1, main () at exp4.c:9
9          sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at exp4.c:9
9          sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10

```

```
(kali@kali)-[~/exp]
$ cc exp4.c -o exp4

(kali@kali)-[~/exp]
$ ./exp4
Sum = 15
```

Explanation:

GDB breakpoints and variable display showed the loop stopped before $i=5$. Changing $i < 5$ to $i \leq 5$ fixed the logic.

Case 5

Code:

```
#include <stdio.h>
int square(int x){ return x*x; }
int main(){
    int n=5;
    int result=square(n);
    printf("Square = %d\n",result);
}
```

Screenshots:

```
Reading symbols from ./exp5...
(gdb) break main
Breakpoint 1 at 0x1156: file exp5.c, line 12.
(gdb) run
Starting program: /home/kali/exp/exp5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at exp5.c:12
12      int result, n = 5;
(gdb) next
14      result = square(n);
(gdb) next
16      printf("Square = %d\n", result);
(gdb) print result
$1 = 25
```

```

(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/exp/exp5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at exp5.c:12
12      int result, n = 5;
(gdb) next
14      result = square(n);
(gdb) step
square (x=5) at exp5.c:6
6      result1 = x * x;
(gdb) print x
$2 = 5
(gdb) next
7      return result1;
(gdb) print result1
$3 = 25

```

Explanation:

The 'next' command executed the function without entering it, while 'step' entered the function and allowed inspection of local variables.

Case 6

Code:

```

#include <stdio.h>
int divide(int a,int b){ return a/b; }
int calculate(int x){ return divide(x,0); }
int main(){ printf("Result: %d",calculate(10)); }

```

Screenshots:

```

(gdb) run
Starting program: /home/kali/exp/exp6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x000055555555147 in divide (a=10, b=0) at exp6.c:5
5      return a / b;
(gdb) backtrace
#0  0x000055555555147 in divide (a=10, b=0) at exp6.c:5
#1  0x000055555555166 in calculate (x=10) at exp6.c:10
#2  0x00005555555517a in main () at exp6.c:15
(gdb) frame 0
#0  0x000055555555147 in divide (a=10, b=0) at exp6.c:5
5      return a / b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x000055555555166 in calculate (x=10) at exp6.c:10
10     return divide(x, 0);
(gdb) print x
$2 = 10

```

```
(kali㉿kali)-[~/exp]
$ cc exp6.c -o exp6

(kali㉿kali)-[~/exp]
$ ./exp6
Result: 5
```

Explanation:

Backtrace identified divide() as the crash location due to division by zero. Changing the divisor fixed the runtime error.

Case 7

Code:

```
#include <stdio.h>
int main(){
    int a[5]={1,2,3,4,5};
    for(int i=0;i<=5;i++)
        printf("%d ",a[i]);
}
```

Screenshots:

```
(gdb) break 10
Breakpoint 1 at 0x114a: file exp7.c, line 10.
(gdb) run
Starting program: /home/kali/exp/exp7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at exp7.c:10
10      scanf("%d", &a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd20
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd30
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd34
```

```
(kali㉿kali)-[~/exp]
$ cc exp7.c -o exp7

(kali㉿kali)-[~/exp]
$ ./exp7
1 2 3 4 5

(kali㉿kali)-[~/exp]
$
```

Explanation:

Array index 5 was outside the valid range 0-4. Changing the loop condition to `i < 5` prevented out-of-bounds access.

Case 8

Code:

```
#include <stdio.h>
int main(){
    int *p=NULL;
    printf("%d", *p);
}
```

Screenshots:

```
(gdb) run
Starting program: /home/kali/exp/exp8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00005555555514d in main () at exp8.c:7
7      *p = 10;
(gdb) print *p
X Cannot access memory at address 0x0
```

```
(kali㉿kali)-[~/exp]
$ cc exp8.c -o exp8

(kali㉿kali)-[~/exp]
$ ./exp8
10
```

Explanation:

Dereferencing a NULL pointer caused a segmentation fault. Assigning the pointer to a valid variable fixed the issue.

Conclusion

The experiment demonstrated syntax errors, compiler warnings, logical errors, runtime errors, array bounds issues and segmentation faults. Compiler diagnostics, printf tracing and GDB were successfully used to identify and fix these problems.

Exercises & Homework Questions with Answers

1. Error Classification: Write a small C program snippet illustrating each of the following error types: Syntax error, Compilation/Type error, Linker error, Runtime error, and Logical error.

Answer:

Syntax Error

```
#include <stdio.h>
int main() {
    printf("Hello")
    return 0;
}
(Missing semicolon)
```

Compilation/Type Error

```
#include <stdio.h>
int main() {
    int a = "Hello";
    return 0;
}
```

Linker Error

```
#include <stdio.h>
void display();
int main() {
    display();
    return 0;
}
```

Runtime Error

```
#include <stdio.h>
int main() {
    int a=10,b=0;
    printf("%d", a/b);
    return 0;
}
```

Logical Error

```
#include <stdio.h>
int main() {
    int sum=0;
    for(int i=1;i<5;i++)
```

```
    sum+=i;
    printf("%d",sum);
    return 0;
}
```

2. GNU Concept: Explain what the GNU project is and how GDB fits into the GNU ecosystem.

Answer:

The GNU Project is a free software initiative started by Richard Stallman in 1983 to develop a complete Unix-like operating system using free software. GDB (GNU Debugger) is part of the GNU toolchain and is used to debug C, C++ and other programs by inspecting variables, breakpoints and program execution.

3. Shell Context: Define what a Shell and Kernel are. Briefly list three common Linux shells (e.g., bash, zsh, fish).

Answer:

Shell: A command-line interface that accepts user commands and passes them to the kernel.

Kernel: The core of the operating system that manages hardware, memory, CPU and system resources.

Common Linux shells: bash, zsh and fish.

4. Research and contrast the differences between: Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Answer:

Breakpoint: Stops execution at a specified line to inspect the program.

Watchpoint: Stops execution when the value of a variable changes.

Catchpoint: Stops execution when a specific event such as an exception or signal occurs.

Tracepoint: Records execution information without stopping the program (supported on some targets).